학습 교안

하네스 엔지니어링 입문 1단계

AI 에이전트에게 맡기되, 규칙과 검증으로 통제하는 생초보를 위한 첫걸음

OpenCode · OpenRouter · Python · pytest · Git

0장 준비 + 1~6장 · Docker 없이 진행하는 입문 실습

강사 김대철

작성일 2026.11.12

실습 환경 Windows + Python + Git + OpenCode

실습 저장소 https://github.com/sinaboro/harness-beginner-tutorial

교안의 명령과 테스트 결과는 Windows PowerShell 에서 실제로 실행해 얻은 것입니다.
AI 의 답변은 실행할 때마다 달라지므로, 교안에는 답변 대신 확인 방법을 실었습니다.

목차

과정 안내 **5**

단계 안내 5

2단계 교시표 5

도커로 실행하기 (선택) 5

0단계 교시표 6

1단계 교시표 6

저장소 구조 7

시작하기 7

원칙 세 가지 7

강사 안내 8

1단계 준비하기 (도커) **9**

1. Docker Desktop 설치 (Windows) 9

2. 저장소 받기와 설정 파일 (Windows) 9

3. 작업 환경 만들기 (Windows, 처음 한 번) 9

4. 컨테이너 들어가기 (Windows) 10

5. OpenRouter 연결 (컨테이너 안) 10

6. 작업 폴더 확인 10

확인 체크리스트 10

자주 만나는 오류 10

1교시. 하네스란 무엇인가 — 같은 부탁, 다른 결과 **12**

학습 목표 12

1. 개념 — 네 가지 낱말 12

2. 따라 하기 13

3. 확인하기 14

4. 자주 만나는 오류 14

5. 확인 문제 15

2장. 요구사항 쓰기 **16**

학습 목표 16

1. 개념 — 요구사항은 "약속"이다 16

2. 따라 하기 17

3. 확인하기 18

4. 자주 만나는 오류 18

5. 확인 문제 18

3장. 작업 지시서 쓰기 **19**

학습 목표 19

1. 개념 — 지시서는 "일을 나눠 주는 쪽지" . . . 19
2. 따라 하기 . . . 19
3. 확인하기 . . . 21
4. 좋은 지시와 나쁜 지시 . . . 21
5. 자주 만나는 오류 . . . 21
6. 확인 문제 . . . 21

4장. 규칙과 권한 세우기 . . . 22

학습 목표 . . . 22
1. 개념 — 부탁과 잠금장치 . . . 22
2. 따라 하기 . . . 22
3. 확인하기 . . . 25
4. 자주 만나는 오류 . . . 25
5. 확인 문제 . . . 25

5장. 검증하고 저장하기 . . . 26

학습 목표 . . . 26
1. 개념 — "돌아가는 것 같다"와 "통과했다"는 다르다 . . . 26
2. 따라 하기 . . . 26
3. 확인하기 . . . 29
4. 자주 만나는 오류 . . . 29
5. 확인 문제 . . . 29

6장. 미니 프로젝트: 할 일 목록 . . . 30

학습 목표 . . . 30
1. 개념 — 지금까지의 고리 . . . 30
2. 따라 하기 . . . 30
3. 확인하기 . . . 33
4. 막혔을 때 . . . 34
5. 확인 문제 . . . 34
다음 단계 . . . 34

부록. 템플릿과 예제 . . . 35

A-1. 요구사항 빈칸 틀 (2장) . . . 35
A-2. 작업 지시서 빈칸 틀 (3장) . . . 36
A-3. 규칙 문서 빈칸 틀 (4장) . . . 37
A-4. 권한 설정 완성본 (4장) . . . 38
A-5. 검증 스크립트 (5장) . . . 39
A-6. 커밋 전 검증 훅 (5장) . . . 39
A-7. 할 일 목록 요구사항 (6장) . . . 40
A-8. 할 일 목록 작업 지시서 (6장) . . . 41
A-9. 할 일 목록 테스트 (6장) . . . 42

A-10. 계산기 예제 (4·5장) 43
A-11. 계산기 테스트 (5장) 44

과정 안내

AI 에이전트에게 개발을 맡기되, 규칙·권한·검증으로 통제하고, 통과한 것만 저장하는 법을 단계별로 배웁니다.

코드를 먼저 배운 뒤에 AI 를 쓰는 과정이 아닙니다. AI 에게 맡기면서 코드와 개발 원리를 함께 이해하는 과정입니다. 그래서 각 단계는 "하네스의 한 요소"를 하나씩 익히도록 설계했습니다.

대상	프로그래밍을 처음 배우는 사람 (코딩 경험이 없어도 됩니다)
준비물	Windows PC, Git, OpenCode, OpenRouter 계정 (1단계부터 Python 추가)
필요 없는 것	Docker, 데이터베이스, 웹 서버 (0·1단계 기준)
에이전트	OpenCode (OpenRouter 모델 사용)

단계 안내

단계	대상	내용	상태
0단계	코드를 전혀 모르는 사람	프로그래밍 언어 없이, HTML·CSS 명함 페이지 한 장을 AI 에게 맡기고 하네스의 원리를 익힌다	**이 저장소에 있음** (`docs/stage0/`)
1단계	생초보	Python 계산기와 할 일 목록 프로그램으로, 테스트와 저장 장치까지 익힌다	**이 저장소에 있음** (`docs/stage1/`)
2단계	초급	도서 대출 목록 프로그램으로, 저장, 상태 규칙, 요구사항 변경, 되돌리기를 익힌다	**이 저장소에 있음** (`docs/stage2/`)
3단계	실무형	3일 과정. FastAPI, PostgreSQL, Docker 까지 연결하는 전체 실습	별도 저장소 harness-crud-lab

2단계 교시표

장	주제	하네스 개념	결과물
준비하기	1단계 도구 확인, 작업 폴더 만들기	도구와 실행 환경	설치 확인
1장	도서관 요구사항 쓰기	생활 규칙을 상태로 바꾸기	도서관 요구사항
2장	파일에 저장하기	상태를 파일로 남기기	책 등록, JSON 저장
3장	대출과 반납	작은 지시로 나눠 맡기기, 상태 규칙	대출과 반납 기능
4장	AI 가 만든 실수 찾기	테스트가 잡는 실수와 못 잡는 실수	빠진 약속을 추가한 테스트
5장	요구사항이 바뀌었을 때	기준은 사람이, 구현은 AI 가	변경 요청서, 대출 기간 기록
6장	잘못된 변경을 되돌리기	`restore` 와 `revert`	되돌린 기록
7장	미니 프로젝트: 도서 대출 목록	1~6장 종합	완성된 프로그램, 과정 기록

도커로 실행하기 (선택)

PC 에 Python 과 OpenCode 를 직접 설치하지 않고, 컨테이너 안에서 0·1·2단계를 할 수 있습니다. 방법은 `docs/docker/00_도커로_실행하기.md` 를 봅니다. 편집은 Windows 의 VS Code 에서, 명령은 컨테이너 안에서 합니다.

0단계 교시표

장	주제	결과물
0장	준비하기 (Git, OpenCode, OpenRouter 연결)	설치 확인
1장	AI 와 하네스 체험	조건 없이 시킨 것과 조건을 붙인 것의 비교
2장	말로 된 요구사항 쓰기	명함 요구사항
3장	작업 지시서 쓰기	여섯 칸 지시서
4장	규칙과 권한 세우기	`AGENTS.md`, 권한 설정 확인
5장	체크리스트로 확인하고 저장하기	체크리스트 확인 후 커밋
6장	미니 프로젝트: 자기소개 명함 페이지	HTML·CSS 명함 페이지, 과정 기록

1단계 교시표

교시	주제	배우는 것	결과물
0장	준비하기	Python, Git, OpenCode, OpenRouter 연결	설치 확인 체크리스트
1장	하네스란 무엇인가	같은 부탁을 조건 없이 한 것과 조건을 붙인 것의 차이	비교 기록
2장	요구사항 쓰기	"이렇게 하면 → 이런 결과"로 흐리지 않게 쓰기	계산기 요구사항
3장	작업 지시서 쓰기	목표·범위·하지 말 것·완료 조건 여섯 칸	지시서와 계획 검토
4장	규칙과 권한 세우기	`AGENTS.md`(부탁)와 `opencode.jsonc`(잠금장치)	규칙 문서, 권한 확인
5장	검증하고 저장하기	빨강→초록, 검증을 통과해야 커밋되는 훅	통과한 커밋
6장	미니 프로젝트	할 일 목록을 AI 에게 맡기고 통제하기	할 일 목록 프로그램, 과정 기록

각 장은 **학습 목표 → 개념 → 따라 하기 → 확인하기 → 자주 만나는 오류 → 확인 문제** 순서입니다.

저장소 구조

```
harness-beginner-tutorial/
├ README.md                 이 문서
├ docs/
|   ├ stage0/               0단계 교안 (0장~6장)
|   ├ stage1/               1단계 교안 (0장~6장)
|   ├ docker/               도커로 실행하기 (0·1·2단계 공통)
|   ├ stage2/               2단계 교안 (준비하기, 1장~7장)
|   ├ 하네스_입문_0단계_교안.pdf
|   ├ 하네스_입문_1단계_교안.pdf
|   └ 하네스_입문_2단계_교안.pdf
├ templates/                수업에서 복사해서 쓰는 틀과 파일
|   ├ stage0/               0단계 틀 (명함 요구사항·지시서·규칙·권한·체크리스트)
|   ├ stage2/               2단계 틀 (도서관 요구사항·지시서·변경 요청서·테스트 기준)
|   ├ REQ_빈칸.md            2장: 요구사항 빈칸 틀
|   ├ TASK_빈칸.md           3장: 작업 지시서 빈칸 틀
|   ├ AGENTS_빈칸.md         4장: 규칙 문서 빈칸 틀
|   ├ opencode.jsonc        4장: 권한 설정 (완성본)
|   ├ REQUIREMENTS.md       6장: 할 일 목록 요구사항
|   ├ TASK_todo.md          6장: 할 일 목록 작업 지시서
|   ├ tests/test_todo.py    6장: 할 일 목록 테스트 (AI 가 고칠 수 없는 기준)
|   ├ verify.py             5장: 검증 스크립트
|   ├ pre-commit            5장: 커밋 전 검증 훅
|   └ .gitattributes        줄바꿈 설정
├ docker/                   도커 실행 환경 (0·1·2단계 공용)
|   ├ Dockerfile            Python, pytest, Git, OpenCode 가 든 이미지 설계도
|   ├ docker-compose.yml    작업 폴더와 템플릿 폴더를 연결하는 설정
|   └ .env.example          경로와 Git 이름 설정 견본
├ examples/
|   ├ calc/                 1단계 4·5장에서 쓰는 계산기 예제 (multiply 가 빠진 상태)
|   └ stage2/               2단계 4장에서 쓰는 연습용 초안 (검증 전, 실수 포함)
└ tools/
    └ build_guide.py        교안 PDF 를 만드는 스크립트 (인자: 0 또는 1)
```

시작하기

0단계(`docs/stage0/`), 1단계(`docs/stage1/`), 2단계(`docs/stage2/`) 순서로 따라 합니다.

■ **[PS] 윈도우 PowerShell** 저장소를 내려받습니다.

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
```

교안은 PDF 로도 볼 수 있습니다. `docs/하네스_입문_0단계_교안.pdf`, `docs/하네스_입문_1단계_교안.pdf`, `docs/하네스_입문_2단계_교안.pdf` 를 여세요.

0단계부터 순서대로 합니다. 0단계는 Python 이 필요 없고, 1단계부터 Python 을 씁니다.

원칙 세 가지

1. **AI 가 쓰고, 사람이 판단합니다.** 코드는 AI 가 쓰더라도, 계획을 승인하고 결과를 읽고 저장하는 판단은 사람이 합니다.
2. **기준은 AI 가 고치지 않습니다.** 테스트와 검증 스크립트, 규칙 문서는 권한 설정으로 AI 가 고치지 못하게 막습니다.
3. **통과한 것만 저장합니다.** 검증이 실패하면 커밋 자체가 취소됩니다.

강사 안내

- 교안의 실행 결과는 0장~6장의 명령을 실제 Windows PowerShell 에서 확인한 것을 기준으로 합니다.
  OpenCode 의 AI 답변은 실행할 때마다 다르므로, 교안에는 답변 내용 대신 **확인 방법**을 적었습니다.
- 6장에는 정답 코드를 싣지 않았습니다. 학생이 테스트와 지시서를 읽고 AI 와 대화하며 맞춰 가도록 한 것입니다.
- 3단계(3일 과정)는 별도 저장소 harness-crud-lab 에 있습니다. 2단계 7장을 마치면 그 저장소로 이어 갑니다.

1단계 준비하기 (도커)

시간	1시간 30분 (처음 한 번)
준비물	Windows 10·11, Docker Desktop, Git for Windows, VS Code, 인터넷, OpenRouter 계정
결과	도커 컨테이너 안에 Python, pytest, Git, OpenCode 가 준비되고, OpenRouter 에 연결된다

1단계부터 Python 을 씁니다. 그래도 **PC 에 Python 을 설치하지 않습니다.** Python 은 컨테이너 안에 들어 있습니다.

> **역할 나누기**
> - **Windows 쪽(PowerShell)**: 설치하고, 저장소를 받고, 컨테이너를 켠다. `[Windows]` 표시.
> - **컨테이너 쪽(bash)**: 교안의 실습 명령을 친다. `[컨테이너]` 표시.
> - **편집과 확인**: Windows 의 VS Code 에서 파일을 고친다.

1. Docker Desktop 설치 (Windows)

1. https://www.docker.com/products/docker-desktop/ 에서 받아 설치합니다.
2. 설치 중 **Use WSL 2 instead of Hyper-V** 가 보이면 체크된 채로 둡니다.
3. PC 를 재시작하고 Docker Desktop 을 실행합니다. **Engine running** 이 될 때까지 기다립니다.

[Windows] (PowerShell)

```
docker version
```

Client 와 Server 두 줄이 모두 나와야 합니다.

2. 저장소 받기와 설정 파일 (Windows)

[Windows] (PowerShell)

```
git clone https://github.com/sinaboro/harness-beginner-tutorial C:\harness-beginner-tutorial
New-Item -ItemType Directory C:\harness-lab
cd C:\harness-beginner-tutorial\docker
Copy-Item .env.example .env
```

VS Code 로 `C:\harness-beginner-tutorial\docker\.env` 를 열어 `GIT_NAME` 과 `GIT_EMAIL` 을 본인 것으로 바꾸고 저장합니다.

3. 작업 환경 만들기 (Windows, 처음 한 번)

[Windows] (PowerShell)

```
cd C:\harness-beginner-tutorial\docker
docker compose build
```

처음에는 몇 분 걸립니다. 마지막에 `Built` 또는 `Successfully` 가 보이면 됩니다.

4. 컨테이너 들어가기 (Windows)

[Windows] (PowerShell)

```
docker compose run --rm workspace
```

프롬프트가 `root@…:/workspace#` 로 바뀌면 컨테이너 안입니다. 확인합니다.

[컨테이너]

```
pwd
python --version
python -m pytest --version
git --version
opencode --version
```

`/workspace` 는 Windows 의 `C:\harness-lab` 과 같은 폴더이고, `/tutorial` 에는 교안의 템플릿과 예제가 있습니다(읽기 전용).

5. OpenRouter 연결 (컨테이너 안)

[컨테이너]

```
opencode
```

입력 칸에 `/connect` 를 치고 **OpenRouter** 를 고른 뒤 키를 붙여 넣습니다. `/models` 로 목록이 나오면 `/exit` 로 나옵니다.

> 키는 도커 볼륨에 남습니다. 다음에 다시 들어가도 연결은 그대로입니다.

6. 작업 폴더 확인

[컨테이너]

```
mkdir -p /workspace/calc
ls -la /workspace
```

Windows 탐색기에서 `C:\harness-lab\calc` 가 보이면 연결이 된 것입니다.

확인 체크리스트

- ■ `docker version` 에 Server 가 나온다 (Windows)
- ■ 컨테이너 안에서 `python --version` 과 `python -m pytest --version` 이 나온다
- ■ 컨테이너 안에서 `opencode` → `/models` 목록이 나온다
- ■ Windows 의 `C:\harness-lab\calc` 가 보인다

자주 만나는 오류

증상	해결
`error during connect`	Docker Desktop 이 Engine running 이 아니다. 기다리거나 재시작한다.
`/workspace` 가 비어 있다	`.env` 의 `HARNESS_LAB_DIR` 이 틀렸다. 슬래시로 쓰고 폴더가 있는지 본다.
`pytest` 가 파일을 못 찾는다	명령을 친 폴더가 맞는지 `pwd` 로 확인한다. 과제 폴더(`/workspace/calc` 등)에서 실행한다.
`git commit` 이 `Please tell me who you are`	`.env` 의 `GIT_NAME`, `GIT_EMAIL` 을 채우고 컨테이너를 다시 들어간다.

자세한 설명은 도커로 실행하기를 봅니다.

1교시. 하네스란 무엇인가 — 같은 부탁, 다른 결과

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	0장 완료 (OpenCode 가 OpenRouter 에 연결된 상태)
결과	하네스가 있을 때와 없을 때의 차이를 내 눈으로 확인한다

학습 목표

1. AI 모델, AI 에이전트, 도구, 하네스를 한 문장씩 구분한다.
2. 같은 부탁을 두 가지 방식으로 해 보고, 결과가 어떻게 달라지는지 기록한다.
3. 하네스의 다섯 요소(규칙, 도구, 권한, 실행, 검증)를 말할 수 있다.

1. 개념 — 네 가지 낱말

낱말	한 문장 설명	비유
AI 모델	글을 받아 글을 돌려준다. 파일을 보지 못하고 명령도 실행하지 못한다.	전화로만 대답하는 똑똑한 상담원
AI 에이전트	모델의 말대로 파일을 읽고 고치고 명령을 실행하는 프로그램. 이 교안에서는 OpenCode.	상담원의 지시를 받아 내 책상에서 일하는 직원
도구	에이전트가 일할 때 쓰는 것. 파일 읽기, 명령 실행, 테스트 실행기 등.	직원이 쓰는 망치와 줄자
하네스	에이전트의 일을 통제하는 장치 전체.	말에게 씌우는 마구(고삐와 안장)

하네스(harness) 는 원래 말에게 씌우는 마구입니다. 말은 힘이 세고 빠르지만, 마구가 없으면 원하는 방향으로 달릴 수 없습니다. 마구는 말의 힘을 줄이는 것이 아니라, 힘을 쓸모 있게 만드는 것입니다. AI 에이전트도 같습니다.

바이브 코딩과 하네스 엔지니어링

	바이브 코딩	하네스 엔지니어링
시키는 방법	"메모 앱 만들어 줘"	요구사항과 작업 지시서를 쓴다
결과의 확인	돌아가는 것 같으면 OK	테스트를 돌려 통과했는지 본다
AI 가 할 수 있는 일	무엇이든	허락한 것만
끝난 뒤	내가 설명할 수 없는 코드가 남는다	읽고 설명할 수 있는 코드가 남는다

하네스의 다섯 요소

요소	질문	이 교안에서 만나는 것
규칙	어떻게 일해야 하나?	`AGENTS.md` (4장)
도구	무엇으로 일하나?	Python, pytest, Git
권한	어디까지 해도 되나?	`opencode.jsonc` (4장)
실행	어디에서 실행되나?	작업 폴더 `/workspace`
검증	다 됐는지 어떻게 아나?	`verify.py` 와 테스트 (5장)

2. 따라 하기

1단계. 실험 폴더 만들기

[컨테이너]

```
mkdir -p /workspace/lesson01
cd /workspace/lesson01
opencode
```

OpenCode 화면이 열립니다. 이 폴더가 AI 가 볼 수 있는 전부입니다.

2단계. 실험 A — 아무 규칙 없이 한 줄로 시키기

OpenCode 입력 칸에 아래를 그대로 붙여 넣고 Enter 를 누릅니다.

■ [AI] OpenCode 입력 칸

```
두 수를 더하고 빼는 계산기 파일 calc.py 를 만들어 줘.
```

AI 가 일을 끝낼 때까지 기다립니다. 그다음 아래 표를 채웁니다. (`/exit` 로 나온 뒤 컨테이너 셸에서 `ls` 과 `type calc.py` 로 확인해도 됩니다.)

관찰할 것	내 기록
고치기 전에 계획을 말했나?	
파일을 몇 개 만들었나? 이름은?	
나눗셈 같은 기능을 추가했나? 시키지 않았는데?	
0으로 나누는 경우를 생각했나?	
테스트 파일을 만들었나?	
실행해서 확인했나?	

`/exit` 로 나온 뒤, 만들어진 파일을 실행해 봅니다.

[컨테이너]

```
python calc.py
```

화면에 아무 일도 없거나 오류가 나도 괜찮습니다. 이 실험의 목적은 **AI 가 무엇을 멋대로 정했는지** 보는 것입니다.

3단계. 실험 B — 같은 목표를, 조건을 붙여서

폴더를 새로 만듭니다.

[컨테이너]

```
mkdir -p /workspace/lesson01b
```

```
cd /workspace/lesson01b
opencode
```

OpenCode 에 이번에는 조건 세 가지를 붙여서 시킵니다.

■ [AI] OpenCode 입력 칸

```
두 수를 더하고 빼는 계산기 파일 calc.py 를 만들어 줘.
조건 1. 파일은 calc.py 하나만 만든다. 다른 파일은 만들지 않는다.
조건 2. 더하기(add)와 빼기(subtract) 함수만 만든다. 다른 기능은 추가하지 않는다.
조건 3. 파일을 고치기 전에 계획을 먼저 말하고, 내가 "진행"이라고 할 때까지 기다린다.
```

AI 가 계획을 말하면 읽어 보고, 괜찮으면 진행 이라고 답합니다. 끝난 뒤 같은 표를 채웁니다. 두 실험의 차이를 아래 표에 적습니다.

관찰할 것	실험 A (한 줄)	실험 B (조건 붙임)
계획을 먼저 말했나?		
파일 개수		
시키지 않은 기능이 있나?		
0으로 나누기를 다뤘나?		

4단계. 차이를 말로 설명하기

실험 B 에서 조건 세 개는 **무엇을 막으려고** 붙인 것인지 생각합니다.

조건	막으려는 것
조건 1 (파일 하나만)	
조건 2 (기능 두 개만)	
조건 3 (계획 먼저)	

조건으로 적은 문장이 바로 **하네스의 규칙(부탁)** 입니다. 4장에서 이런 문장을 `AGENTS.md` 로 옮겨 적습니다.

3. 확인하기

- ■ 실험 A 와 B 의 결과 차이를 두 가지 이상 적었다.
- ■ 조건 세 개가 각각 무엇을 막는지 말할 수 있다.
- ■ 하네스의 다섯 요소를 순서대로 말할 수 있다.

4. 자주 만나는 오류

증상	해결
`opencode` 가 실행되지 않는다	컨테이너 셸에서 실행하는지 확인한다.
AI 가 "모델을 선택하세요"라고 한다	`/connect` 로 OpenRouter 키를 연결한다(0장 5절).
AI 가 계획 없이 바로 파일을 만든다	조건 3 을 다시 붙여 넣는다. 이 단계에서는 규칙이 없으니 정상입니다.
`python calc.py` 에서 오류가 난다	오류 내용을 적어 둔다. 2장과 5장에서 이런 오류를 다룹니다.

5. 확인 문제

1. AI 모델과 AI 에이전트의 차이를 "파일"이라는 낱말을 넣어 설명하시오.
2. 실험 A 에서 AI 가 시키지 않은 것을 했다면, 무엇이었나?
3. 조건 3(계획 먼저)이 없으면 어떤 위험이 생기는가?
4. 하네스의 다섯 요소를 쓰시오.
5. 같은 부탁을 여러 번 해도 결과가 똑같지 않은 이유를 한 문장으로 쓰시오.

2장. 요구사항 쓰기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1장 완료. VS Code 또는 VS Code
결과	"이렇게 부르면 → 이런 결과" 형식으로 요구사항 한 장을 쓴다

학습 목표

1. 좋은 요구사항과 나쁜 요구사항을 구분한다.
2. 요구사항의 세 부분(이렇게 하면, 이런 결과가 나온다, 잘못 쓰면)을 채운다.
3. 모호한 낱말을 찾아 눈으로 확인할 수 있는 말로 바꾼다.

1. 개념 — 요구사항은 "약속"이다

요구사항은 "무엇을 만들지"를 사람끼리, 그리고 AI 와 약속하는 글입니다. 약속이 흐리면 AI 는 빈 곳을 제멋대로 채웁니다. 1장의 실험 A 에서 AI 가 나눗셈을 추가한 것도 약속이 없었기 때문입니다.

좋은 요구사항과 나쁜 요구사항

나쁜 요구사항	왜 나쁜가	좋은 요구사항
계산기를 만든다	무슨 계산인지 모른다	두 수를 받아 더한 결과를 돌려준다
빠르게 계산한다	"빠르게"를 잴 방법이 없다	계산 결과는 즉시 화면에 나온다 (확인 방법: 실행하면 바로 답이 나온다)
0 나누기도 잘 처리한다	"잘"이 무엇인지 모른다	0으로 나누면 "0으로 나눌 수 없습니다."라는 오류를 낸다
적당히 예쁘게 만든다	"적당히", "예쁘게"는 사람마다 다르다	(이번 교안에서는 화면을 만들지 않는다)

요구사항의 세 칸

칸	질문	예
이렇게 하면	무엇을 어떻게 부르나?	`add(2, 3)` 을 부른다
이런 결과가 나온다	무엇이 돌아오나?	`5` 가 돌아온다
잘못 쓰면	잘못된 입력일 때는?	`divide(1, 0)` 이면 `ValueError` 오류가 난다

"잘못 쓰면" 칸이 가장 자주 빠집니다. 빠진 칸은 AI 가 마음대로 정합니다.

2. 따라 하기

1단계. 나쁜 요구사항 고치기 (20분)

아래 문장을 읽고, 흐린 낱말에 밑줄을 긋습니다.

> 계산기는 숫자를 잘 계산하고, 빠르고 정확하며, 사용하기 편해야 한다.

흐린 낱말은 "잘", "빠르고", "정확하며", "편해야"입니다. 각각을 확인할 수 있는 문장으로 바꿉니다.

흐린 낱말	확인할 수 있는 문장으로 바꾸면
잘 계산한다	두 수 2와 3을 더하면 5가 나온다
빠르게	
정확하게	
편하게	

> 힌트: "편하게"는 쉽지 않습니다. 이번 교안에서는 화면을 만들지 않으므로, "편하게"를 요구사항에서 빼는 것도 답입니다. 빼는 것이 맞는 경우도 있습니다.

2단계. 요구사항 틀 채우기 (40분)

저장소의 빈칸 틀을 작업 폴더에 복사합니다.

[컨테이너]

```
mkdir -p /workspace/요구사항연습
cp /tutorial/templates/REQ_빈칸.md /workspace/요구사항연습/요구사항.md
# (Windows) /workspace/요구사항연습/요구사항.md 을(를) VS Code 로 열어 고칩니다
```

VS Code에서 계산기 요구사항을 채웁니다. 조건은 다음과 같습니다.

- 기능은 **세 가지**만 씁니다: 더하기, 빼기, 나누기
- 나누기에는 반드시 "잘못 쓰면" 칸을 채웁니다.
- "만들지 않는 것"에 두 가지 이상을 적습니다. (예: 곱하기, 화면, 파일 저장)

3단계. 점검하기 (20분)

다 쓴 요구사항을 아래 질문으로 점검합니다. 하나라도 "아니오"면 고칩니다.

- ■ 옆 사람이 읽고 똑같은 계산기를 만들 수 있는가?
- ■ 모든 기능에 "이렇게 하면 / 이런 결과 / 잘못 쓰면"이 있는가?
- ■ 결과를 눈으로 확인할 수 있는가? ("잘"이나 "편하게"가 없는가?)
- ■ "만들지 않는 것"이 적혀 있는가?

4단계. 다른 사람과 바꿔 읽기 (20분)

옆 사람의 요구사항을 읽고, **읽는 사람 입장에서** 질문을 한 개씩 적어 줍니다. 질문이 많이 나오면 요구사항이 약하다는 뜻입니다.

예: "0으로 나누면 오류 메시지를 정확히 어떤 글자로 내나요?" → 이 질문에 답이 적혀 있지 않다면 고칩니다.

5단계. 예제와 비교하기 (10분)

저장소의 예제 코드를 읽고, 요구사항의 한 줄이 코드의 어디에 있는지 찾습니다.

[컨테이너]

```
type /tutorial/examples/calc/calc.py
```

- `add` 함수 위의 설명 `"""두 수를 더한다."""` 는 요구사항의 어느 줄에 해당하나?
- `divide` 의 `if b == 0:` 은 요구사항의 어느 줄에 해당하나?

요구사항이 코드가 되는 과정을 눈으로 본 것입니다. 이 연결이 **테스트**가 됩니다(5장).

3. 확인하기

- ■ 계산기 요구사항을 세 기능과 "잘못 쓰면" 칸까지 채웠다.
- ■ "만들지 않는 것"을 두 가지 이상 적었다.
- ■ 옆 사람의 요구사항에서 질문을 하나 이상 찾았다.

4. 자주 만나는 오류

증상	해결
요구사항에 "잘", "적당히"가 계속 나온다	1단계의 표로 돌아가 낱말을 확인 가능한 문장으로 바꾼다.
"이런 결과"를 어떻게 쓸지 모르겠다	계산 결과를 숫자로 적는다. 예: `2와 3을 더하면 5`
기능이 너무 많다	세 기능만 남기고 나머지는 "만들지 않는 것"으로 옮긴다.

5. 확인 문제

1. "빠르게 계산한다"를 확인할 수 있는 문장으로 바꾸시오.
2. "잘못 쓰면" 칸이 왜 중요한가? 칸이 비어 있으면 AI 에게 어떤 일이 생기는가?
3. "만들지 않는 것"을 적는 이유를 한 문장으로 쓰시오.
4. 요구사항에 "어떻게 만들지"(예: 리스트를 쓴다)를 적으면 안 되는 이유는 무엇인가?

3장. 작업 지시서 쓰기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	2장의 요구사항, OpenCode 연결 완료
결과	지시서 여섯 칸을 채워 AI 에게 건네고, 계획을 검토해 승인한다

학습 목표

1. 작업 지시서의 여섯 칸(목표, 요구사항, 범위, 하지 말 것, 완료 조건, 진행 방법)을 설명한다.
2. 한 줄 지시와 지시서 지시의 결과를 비교한다.
3. AI 의 계획을 검토하고, 맞지 않으면 고치게 한다.

1. 개념 — 지시서는 "일을 나눠 주는 쪽지"

직장에서 일을 맡길 때 "알아서 해 주세요"라고만 하면 결과가 엉뚱하게 나옵니다. 목표, 범위, 하지 말 것, 끝났다는 기준을 말해 줘야 합니다. 작업 지시서는 그 내용을 글로 적은 것입니다.

칸	질문	빠지면 생기는 일
1. 목표	끝나면 무엇이 되어 있나?	어디까지가 끝인지 모른다
2. 요구사항	어느 번호를 구현하나?	기능의 모양을 AI 가 지어낸다
3. 범위	어떤 파일을 고쳐도 되나?	관계없는 파일이 바뀐다
4. 하지 말 것	무엇을 덧붙이면 안 되나?	시키지 않은 기능이 생긴다
5. 완료 조건	끝났다는 것을 어떻게 확인하나?	"다 했습니다"를 믿을 수밖에 없다
6. 진행 방법	계획을 먼저 말하는가?	방향이 틀려도 끝난 뒤에야 안다

확인할 수 있는 문장으로 쓴다. "잘 동작한다"는 확인할 수 없습니다. "`python verify.py` 가 검증 통과로 끝난다"는 확인할 수 있습니다.

2. 따라 하기

1단계. 빈칸 틀 복사하기 (5분)

[컨테이너]

```
mkdir -p /workspace/지시서연습
cp /tutorial/templates/TASK_빈칸.md /workspace/지시서연습/지시서.md
# (Windows) /workspace/지시서연습/지시서.md 을(를) VS Code 로 열어 고칩니다
```

2단계. 지시서 여섯 칸 채우기 (30분)

2장에서 쓴 계산기 요구사항을 바탕으로 채웁니다. 조건은 다음과 같습니다.

- 1번 목표: 계산기 파일 하나를 만든다.

- 2번 요구사항: 2장에서 정한 번호를 적는다.
- 3번 범위: 고쳐도 되는 파일은 `calc.py` 하나로 적는다.
- 4번 하지 말 것: 최소 두 가지를 적는다. (예: 화면, 파일 저장)
- 5번 완료 조건: "calc.py 를 `python calc.py` 로 실행하면 오류 없이 끝난다"와, 요구사항의 결과 두 개 이상을 적는다.
- 6번 진행 방법: 계획을 먼저 말하고 승인을 기다린다는 문장을 그대로 둔다.

3단계. 한 줄 지시와 비교하기 (20분)

1장의 실험 A 를 떠올립니다. 이번에는 지시서를 통째로 AI 에게 건넵니다. 폴더를 새로 만듭니다.

[컨테이너]

```
mkdir -p /workspace/calc3
cd /workspace/calc3
opencode
```

OpenCode 입력 칸에 다음을 붙여 넣습니다. `지시서.md` 의 내용을 그대로 붙여 넣어도 되고, 파일을 가리켜도 됩니다.

■ [AI] OpenCode 입력 칸

```
/workspace/지시서연습/지시서.md 를 읽고 그 지시대로 작업해 줘. 먼저 계획만 말하고 내 승인을 기다려.
```

> 파일 경로를 읽지 못하면, 지시서 내용을 직접 붙여 넣습니다.

4단계. 계획 검토하기 (20분)

AI 가 말한 계획을 아래 체크리스트로 검토합니다.

■ 고칠 파일이 범위(3번)에 적힌 파일뿐인가?
■ "하지 말 것"(4번)에 적힌 기능을 계획에 넣지 않았나?
■ 완료 조건(5번)을 확인하는 방법을 말했나?
■ 모르는 낱말이 있나? 있으면 지금 묻는다.

맞지 않으면 구체적으로 짚어서 고치게 합니다.

■ [AI] OpenCode 입력 칸

```
계획 2번에 나눗셈이 있어. 지시서 4번에서 "하지 말 것"에 적은 것과 맞지 않아. 빼고 계획을 다시 말해 줘.
```

맞으면 승인합니다.

■ [AI] OpenCode 입력 칸

```
진행
```

5단계. 결과 확인하기 (20분)

AI 가 일을 끝내면 지시서 5번(완료 조건)을 하나씩 직접 확인합니다.

[컨테이너]

```
cd /workspace/calc3
ls -la
python calc.py
```

■ 파일이 지시서 3번 범위 안에 있다.
■ 4번에 적은 기능이 없다.

■ 5번의 결과가 실제로 나온다.

6단계. 한 줄 지시와 비교하기 (15분)

1장 실험 A 의 기록과 이번 결과를 나란히 놓고 표를 채웁니다.

항목	1장 실험 A (한 줄)	3장 (지시서)
계획을 먼저 말했나?		
시키지 않은 기능이 있나?		
완료 조건을 확인할 수 있었나?		
내가 결과를 판단하기 쉬웠나?		

3. 확인하기

■ 지시서 여섯 칸을 모두 채웠다.

■ AI 의 계획에서 빠진 부분이나 넘친 부분을 하나 이상 찾아 고치게 했다.

■ 5번 완료 조건을 내 눈으로 확인했다.

4. 좋은 지시와 나쁜 지시

나쁜 지시	왜 나쁜가	좋은 지시
"계산기 만들어 줘"	목표와 범위가 없다	"지시서대로 calc.py 를 만든다. 더하기와 빼기만 만든다."
"에러 고쳐 줘"	어떤 에러인지 모른다	"`python calc.py` 를 실행하면 이 오류가 난다. 원인을 먼저 설명하고 고쳐 줘."
"알아서 잘 해 줘"	판단을 모두 AI 에게 넘긴다	"두 방법이 있으면 장단점을 말하고 내가 고르게 해 줘."

5. 자주 만나는 오류

증상	해결
AI 가 지시서 파일을 못 찾는다	파일 경로를 확인하고, 내용을 직접 붙여 넣는다.
계획 없이 바로 파일을 고친다	"진행" 전에는 파일을 고치지 말라고 다시 말한다. 이미 고쳤다면 `/undo` 를 쓴다.
지시서에 없는 기능이 들어간다	4번 "하지 말 것"을 가리키며 빼게 한다.

6. 확인 문제

1. 작업 지시서의 여섯 칸을 순서대로 말하시오.
2. "완료 조건"을 "잘 동작한다"로 쓰면 어떤 문제가 생기는가?
3. "범위"와 "하지 말 것"의 차이를 설명하시오.
4. AI 에게 "먼저 계획만 말해"라고 하는 이유는 무엇인가?

4장. 규칙과 권한 세우기

실행 환경: 도커. 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	3장 완료, 빈칸 틀(`AGENTS_빈칸.md`), 권한 설정(`opencode.jsonc`)
결과	`AGENTS.md` 를 채우고, 권한을 걸어 막히는 일과 물어보는 일을 직접 확인한다

학습 목표

1. **규칙(부탁)** 과 **권한(잠금장치)** 의 차이를 설명한다.
2. `AGENTS.md` 에 꼭 들어가야 할 문장을 채운다.
3. `opencode.jsonc` 의 규칙을 읽고, 허용·질문·금지가 실제로 어떻게 작동하는지 확인한다.

1. 개념 — 부탁과 잠금장치

3장에서는 지시서를 매번 건넸습니다. 같은 내용을 매번 말하지 않으려고 규칙을 파일로 적어 둡니다.

	규칙 (`AGENTS.md`)	권한 (`opencode.jsonc`)
성격	부탁	잠금장치
어떻게 작동하나	AI 가 읽고 따른다	금지된 일은 실행 자체가 막힌다
잊거나 어기면?	어길 수 있다	막힌다
예	"커밋은 하지 않는다"	`git commit` 을 deny 로 막는다

교실 문에 "들어가지 마시오"라고 써 붙이는 것(규칙)과, 문을 잠그는 것(권한)의 차이입니다. 둘 다 있어야 합니다. 써 붙여야 이유를 알고, 잠가야 확실히 막힙니다.

권한 규칙 읽는 법

```
{ "action": "shell", "resource": "git commit *", "effect": "deny" }
```

칸	뜻	예
action	무엇을 하려는가	`shell`(명령 실행), `edit`(파일 고치기), `read`(파일 읽기)
resource	무엇에 대해서인가	`git commit *` (`*` 는 "아무 글자나")
effect	어떻게 처리하나	`allow`(그냥 허용), `ask`(나에게 묻기), `deny`(금지)

규칙은 위에서 아래로 읽고, 가장 아래에 걸린 규칙이 이깁니다. 그래서 넓은 규칙은 위에, 예외는 아래에 씁니다.

2. 따라 하기

1단계. 프로젝트 폴더 준비하기 (10분)

5장에서도 이 프로젝트를 씁니다. 예제 계산기를 작업 폴더로 복사하고, 권한 설정과 검증 파일을 함께 넣습니다.

[컨테이너]

```
cp -r /tutorial/examples/calc /workspace/calc
cp /tutorial/templates/verify.py /workspace/calc/
cp /tutorial/templates/opencode.jsonc /workspace/calc/
cp /tutorial/templates/AGENTS_빈칸.md /workspace/calc/AGENTS.md
cd /workspace/calc
ls -la
```

`calc.py`, `tests`, `pytest.ini`, `verify.py`, `opencode.jsonc`, `AGENTS.md` 가 보이면 됩니다.

2단계. AGENTS.md 채우기 (40분)

VS Code 로 `AGENTS.md` 를 엽니다.

[Windows]

```
# (Windows) /workspace/calc/AGENTS.md 을(를) VS Code 로 엽니다
```

`[작성]` 으로 표시된 곳을 채웁니다. 아래 안내를 따릅니다.

절	무엇을 적나	예
1. 이 프로젝트는 무엇인가	목표와 사용자	목표: 더하기·빼기·나누기 계산기. 사용자: 프로그래밍을 처음 배우는 사람
2. 파일 나누기	고쳐도 되는 파일	`calc.py` 는 고쳐도 된다. `tests/` 는 고치면 안 된다.
3. 일하는 순서	계획, 승인, 구현, 검증, 보고	계획을 말하고 "진행"을 기다린 뒤 구현한다.
4. 하지 말 것	커밋, 푸시, 테스트 변경	커밋과 푸시는 사람이 한다. 테스트 기대값을 바꾸지 않는다.
5. 코드 규칙	주석과 이름 짓기	모든 함수에 한국어 설명을 단다.
6. 검증	완료라고 말하기 전 실행할 명령	`python verify.py`
7. 보고 형식	보고에 들어갈 것	한 일, 바뀐 파일, 검증 결과, 확인 방법

채운 뒤 체크리스트. 아래 다섯 문장이 모두 들어 있어야 합니다. 빠진 것이 있으면 추가합니다.

- ■ 계획을 먼저 말하고, 사람의 "진행"을 기다린다는 문장
- ■ 테스트 파일을 고치지 않는다는 문장
- ■ 커밋과 푸시는 하지 않는다는 문장
- ■ `python verify.py` 로 검증한다는 문장
- ■ 바뀐 파일과 확인 방법을 보고한다는 문장

3단계. 권한 설정 읽기 (15분)

`opencode.jsonc` 를 엽니다.

[Windows]

```
# (Windows) /workspace/calc/opencode.jsonc 을(를) VS Code 로 엽니다
```

아래 질문에 답합니다.

1. `python verify.py` 를 AI 가 실행하려 하면 묻지 않고 실행되나? (allow / ask / deny)
2. `git commit` 을 실행하려 하면? (allow / ask / deny)

3. `calc.py` 를 고치려 하면? (힌트: `edit` 규칙 중 `calc.py` 에 걸리는 것이 있나?)
4. `tests/test_calc.py` 를 고치려 하면?
5. `.env` 파일을 읽으려 하면?

> 답: 1 allow, 2 deny, 3 걸리는 규칙이 없으니 기본값(허용), 4 deny, 5 deny

4단계. 권한이 실제로 막는지 확인하기 (30분)

이제 AI 에게 막혀야 할 일을 일부러 시킵니다. 먼저 연습용 `.env` 파일을 만듭니다. 값은 실제 키가 아닌 연습용 문자열입니다.

[컨테이너]

```
cd /workspace/calc
echo "TEST_SECRET=연습용-값입니다" > .env
git init
opencode
```

OpenCode 에서 아래를 하나씩 시키고, 결과를 표에 적습니다.

■ [AI] OpenCode 입력 칸

```
.env 파일을 읽고 안에 무슨 변수가 있는지 알려 줘.
```

■ [AI] OpenCode 입력 칸

```
git commit -m "test" 를 실행해 줘.
```

■ [AI] OpenCode 입력 칸

```
tests/test_calc.py 의 첫 줄에 주석을 한 줄 추가해 줘.
```

■ [AI] OpenCode 입력 칸

```
python --version 명령을 실행해 줘.
```

시킨 일	기대하는 결과	실제 결과
`.env` 읽기	막힌다 (deny)	
`git commit` 실행	막힌다 (deny)	
`tests/` 고치기	막힌다 (deny)	
`python --version` 실행	나에게 묻는다 (ask)	

`ask` 에 걸리면 OpenCode 가 허락을 구합니다. 이때 고를 수 있는 답은 다음과 같습니다.

답	뜻
once	이번 한 번만 허락
always	앞으로 같은 요청은 묻지 않음 (이 교안에서는 쓰지 않는다)
reject	허락하지 않음

이 교안에서는 `once` 또는 `reject` 만 고릅니다.

`/exit` 로 나온 뒤 확인합니다.

[컨테이너]

```
git status
git log --oneline
```

커밋이 생기지 않았고, `tests/test_calc.py` 가 바뀌지 않았다면 권한이 제대로 작동한 것입니다. 기대와 다르면 강사에게 알려 주세요.

5단계. 규칙은 어떻게 작동하나 (15분)

`AGENTS.md` 를 맨 위 폴더에 두면 OpenCode 는 시작할 때 읽습니다. 규칙이 실제로 작동하는지 확인합니다.

OpenCode 를 새로 열고, 규칙에 적은 내용을 **말하지 않고** 물어봅니다.

■ [AI] OpenCode 입력 칸

> 이 프로젝트에서 작업할 때 지켜야 할 규칙이 뭐야? 파일을 고치지 말고 말로만 알려 줘.

AI 가 `AGENTS.md` 의 내용을 말하는지 확인합니다. 말하지 못하면 파일 위치나 이름을 다시 확인합니다.

3. 확인하기

- ■ `AGENTS.md` 에 체크리스트 다섯 문장이 모두 들어 있다.
- ■ `opencode.jsonc` 의 규칙 다섯 개를 말로 설명할 수 있다.
- ■ 막혀야 할 일 세 가지가 실제로 막혔다. (`git status` 로 확인)
- ■ 규칙(부탁)과 권한(잠금)의 차이를 한 문장으로 말할 수 있다.

4. 자주 만나는 오류

증상	해결
OpenCode 가 `AGENTS.md` 를 읽지 않는다	파일이 `/workspace/calc` 맨 위에 있는지 확인한다. `/exit` 후 다시 연다.
설정을 바꿨는데 막히지 않는다	OpenCode 를 새로 열었는지 확인한다. `opencode.jsonc` 의 쉼표와 따옴표를 확인한다.
`git init` 이 안 된다	Git 설치를 확인한다(`git --version`).
`.env` 를 AI 가 읽어서 내용을 보여 준다	권한 설정이 적용되지 않은 것이다. 설정 파일이 `calc` 폴더에 있는지 확인하고 OpenCode 를 다시 연다. 그래도 안 되면 강사에게 알린다.

5. 확인 문제

1. 규칙과 권한의 차이를 "교실 문"이 아닌 다른 비유로 설명하시오.
2. `{ "action": "shell", "resource": "*", "effect": "ask" }` 를 맨 아래로 옮기면 어떻게 되는가?
3. AI 가 `verify.py` 를 고칠 수 있다면, "검증 통과"라는 말을 믿을 수 있는가? 이유를 쓰시오.
4. 이 교안에서 `always` 를 고르지 않는 이유는 무엇인가?

5장. 검증하고 저장하기

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	4장의 `/workspace/calc` 폴더 (AGENTS.md, opencode.jsonc, verify.py 포함)
결과	테스트가 빨강에서 초록으로 바뀌는 것을 보고, 검증을 통과해야만 커밋되게 장치를 켠다

학습 목표

1. 테스트는 "약속"이며, 실행하면 약속을 지켰는지 알려 준다는 것을 설명한다.
2. 빨강(실패)과 초록(통과)의 의미를 말하고, 실패 출력을 읽는다.
3. 검증이 실패하면 커밋이 취소되는 것을 직접 본다.

1. 개념 — "돌아가는 것 같다"와 "통과했다"는 다르다

1장에서 AI 가 만든 계산기가 돌아가는 것처럼 보였습니다. 그러나 나눗셈에서 0으로 나누면 어떻게 되는지, 음수를 넣으면 어떤지는 확인하지 않았습니다. **화면에서 한 번 눌러 본 것은 확인이 아닙니다.**

테스트는 확인을 **코드로 적어 둔 것**입니다.

```
def test_add():
    assert add(2, 3) == 5
```

- `assert` 뒤의 식이 **참**이면 통과(초록),
- **거짓**이면 실패(빨강)입니다.

빨강 → 초록 순서

```
1. 약속(테스트)을 먼저 본다          → 빨강: 아직 기능이 없다
2. AI 에게 기능을 만들게 한다        → 구현
3. 테스트를 다시 실행한다            → 초록: 약속을 지켰다
```

테스트를 고쳐서 초록으로 만들면 안 됩니다. 기준을 바꾸면 검증이 아닙니다.

2. 따라 하기

1단계. 정리하기 (10분)

4장에서 만든 연습용 `.env` 파일은 저장소에 올라가면 안 됩니다. 먼저 치웁니다.

[컨테이너]

```
cd /workspace/calc
rm -f .env
echo ".env" > .gitignore
git status
```

`git status` 에 `.gitignore` 만 새 파일로 보여야 합니다. 4장에서 AI 가 `tests/test_calc.py` 를 고쳤다면 되돌립니다. `git restore tests/` 를 쓰고, 그래도 바뀌어 있으면 강사에게 알립니다.

2단계. 빨강 확인하기 (15분)

테스트를 실행합니다.

[컨테이너]

```
python -m pytest
```

아래와 비슷한 빨간 출력이 나옵니다.

```
ERROR tests/test_calc.py
E   ImportError: cannot import name 'multiply' from 'calc'
```

읽는 법

- `ERROR tests/test_calc.py` → 어느 파일에서 실패했나?
- `ImportError: cannot import name 'multiply'` → 무엇이 없어서 실패했나?

테스트 파일에는 `multiply` 를 쓰는 약속이 있지만, `calc.py` 에는 아직 `multiply` 가 없습니다. 이것이 **빨강**입니다.

> `python -m pytest` 와 `python verify.py` 는 같은 테스트를 실행합니다. `verify.py` 는 결과를 "검증 통과" 또는 "검증 실패"라는 한 줄로 알려 줍니다.

3단계. AI 에게 곱하기 추가시키기 (30분)

지시서를 짧게 씁니다. 테스트를 고치면 안 된다는 조건을 꼭 넣습니다.

[컨테이너]

```
cd /workspace/calc
opencode
```

■ **[AI] OpenCode 입력 칸**

```
calc.py 에 multiply(a, b) 함수를 추가해 줘. 두 수를 곱한 결과를 돌려준다.
규칙:
1. 먼저 계획만 말하고, 내가 "진행"이라고 할 때까지 파일을 고치지 마.
2. tests/ 폴더와 verify.py 는 절대 고치지 마.
3. 끝나면 python verify.py 를 실행해서 결과를 보고해 줘.
```

AI 의 계획을 확인하고 `진행` 이라고 답합니다. 끝나면 `/exit` 로 나옵니다.

[컨테이너]

```
python verify.py
git status
```

■ `python verify.py` 가 "검증 통과"로 끝난다.

■ `git status` 에 `tests/` 가 바뀐 것으로 나오지 않는다.

■ `calc.py` 에 `multiply` 함수가 있고, 함수 위에 한국어 설명이 있다.

> 테스트 기대값(`assert multiply(4, 5) == 20`)을 AI 가 바꾸지 않았는지 꼭 봅니다. 아직 한 번도 저장하지 않았으므로 `git diff` 로는 볼 수 없습니다. 대신 원본과 비교합니다. 아무것도 나오지 않으면 같은 것입니다.
>
> **[컨테이너]**
>
> `bash diff tests/test_calc.py /tutorial/examples/calc/tests/test_calc.py`

4단계. 검증이 저장을 막는 장치 켜기 (20분)

이제 커밋하기 전에 검증을 자동으로 실행하는 장치(훅)를 설치합니다.

[컨테이너]

```
mkdir -p .githooks
cp /tutorial/templates/pre-commit .githooks/pre-commit
git config core.hooksPath .githooks
git config core.hooksPath
```

마지막 명령이 `.githooks` 를 출력하면 켜진 것입니다.

> 훅은 Git 이 `sh` 로 실행합니다. 컨테이너 안에 `sh`(bash)와 파이썬이 들어 있으므로 따로 설치할 것은 없습니다.

5단계. 일부러 깨뜨려 보고, 저장이 막히는 것 보기 (20분)

`calc.py` 의 `divide` 함수를 일부러 망가뜨립니다. VS Code 로 열어 `return a / b` 를 `return a + b` 로 바꾸고 저장합니다.

[Windows]

```
# (Windows) calc.py 을(를) VS Code 로 열어 고칩니다
```

그다음 저장을 시도합니다.

[컨테이너]

```
git add -A
git commit -m "실험: 깨진 코드 저장 시도"
```

아래와 비슷한 출력이 나오고, 커밋이 취소되어야 합니다.

```
[pre-commit] 저장하기 전에 검증을 실행합니다...
...
FAILED tests/test_calc.py::test_divide - assert 12.0 == 3
검증 실패: 위의 오류를 고친 뒤 다시 실행하세요.
[pre-commit] 검증 실패: 커밋을 취소했습니다.
```

커밋이 생기지 않았는지 확인합니다.

[컨테이너]

```
git log --oneline
```

커밋 기록이 없어야 합니다. (아직 처음 커밋이므로 `does not have any commits yet` 메시지가 나와도 정상입니다.)

6단계. 고치고 정식으로 저장하기 (15분)

`divide` 를 원래대로 `return a / b` 로 되돌립니다. VS Code에서 고쳐도 되고, AI 에게 시켜도 됩니다.

[컨테이너]

```
python verify.py
```

"검증 통과"가 나오면 저장합니다.

[컨테이너]

```
git add -A
git commit -m "5장: multiply 추가, 테스트 통과 (검증 통과 후 저장)"
git log --oneline
```

이번에는 `[pre-commit]` 이 통과하고, 커밋 기록이 한 줄 생겼습니다.

3. 확인하기

- 빨강(ImportError)을 보고, 원인을 한 문장으로 말할 수 있다.
- AI 가 `multiply` 를 추가한 뒤 `python verify.py` 가 "검증 통과"로 끝났다.
- 깨뜨린 코드를 커밋하려 했을 때 취소되는 것을 봤다.
- 정식 커밋이 한 줄 생겼다.

4. 자주 만나는 오류

증상	해결
`python -m pytest` 가 `No module named pytest`	0장의 `python -m pip install pytest` 를 다시 실행한다.
`[pre-commit]` 이 나오지 않는다	`git config core.hooksPath` 가 `.githooks` 를 출력하는지 확인한다.
훅이 `python: command not found`	Python 이 PATH 에 없다. 0장을 다시 확인한다.
`LF will be replaced by CRLF` 경고가 뜬다	기능에는 문제가 없다. 템플릿의 `.gitattributes` 를 `calc` 폴더에 복사하면 사라진다.
`git commit` 이 `Please tell me who you are`	0장 6절의 `git config --global` 두 줄을 다시 실행한다.

5. 확인 문제

1. 테스트가 "약속"이라는 말을, `assert` 한 줄을 예로 들어 설명하시오.
2. 빨강에서 초록으로 가는 동안 **테스트 파일을 고치면 안 되는** 이유는 무엇인가?
3. 5단계에서 커밋이 취소된 이유를, 출력에 나온 `FAILED` 줄을 보고 설명하시오.
4. `verify.py` 가 실패하면 종료 코드 1 을 내는 이유는 무엇인가? (힌트: 훅이 이 값을 본다)
5. "화면에서 한 번 눌러 봤다"와 "테스트가 통과했다"의 차이를 말하시오.

6장. 미니 프로젝트: 할 일 목록

> **실행 환경: 도커.** 명령 블록은 컨테이너 안(bash)에서 칩니다. `[Windows]` 로 표시된 블록만 Windows PowerShell 에서 칩니다. 파일 편집은 Windows 의 VS Code 에서 합니다. 자세한 설명은 도커로 실행하기를 봅니다.

시간	2시간
준비물	1~5장 완료. 템플릿(`templates/`) 과 요구사항, 지시서가 준비되어 있다
결과	할 일 목록 프로그램을 AI 에게 맡기고, 검증을 통과한 뒤 저장한다. 과정 기록을 쓴다.

학습 목표

1. 지금까지 만든 다섯 가지(요구사항, 지시서, 규칙, 권한, 검증)를 한 프로젝트에 모두 쓴다.
2. 테스트가 정한 약속을 AI 가 지키게 만든다.
3. 저장 전에 검증이 통과했는지, 테스트 기준이 바뀌지 않았는지 직접 확인한다.
4. 내가 설명할 수 있는 코드만 저장한다.

1. 개념 — 지금까지의 고리

```
① 요구사항 (REQUIREMENTS.md)     무엇을 만들지 약속한다          ← 2장
② 작업 지시서 (TASK_todo.md)     어디까지 맡길지 정한다          ← 3장
③ 규칙과 권한 (AGENTS.md, opencode.jsonc)  AI 의 행동 범위를 정한다  ← 4장
④ 실행과 검증 (tests/, verify.py)    맞는지 확인한다             ← 5장
⑤ 저장 (pre-commit, git commit)      통과한 것만 저장한다          ← 5장
```

이번 장에서 **사람이 하는 일**은 ①과 ②를 준비하는 것(템플릿이 해 줍니다), 계획 승인, 결과 검토, 저장입니다. 코드는 AI 가 씁니다.

2. 따라 하기

1단계. 미니 프로젝트 폴더 만들기 (15분)

[컨테이너]

```
mkdir -p /workspace/my-todo
cd /workspace/my-todo
mkdir -p tests
mkdir -p .githooks
```

템플릿을 가져옵니다.

[컨테이너]

```
T="/tutorial/templates"
cp "$T/tests/test_todo.py" tests/
cp "$T/verify.py" .
cp "$T/opencode.jsonc" .
cp "$T/REQUIREMENTS.md" .
cp "$T/TASK_todo.md" .
cp "$T/pre-commit" .githooks/pre-commit
cp "$T/.gitattributes" .
cp "/tutorial/examples/calc/pytest.ini" .
cp "$T/AGENTS_빈칸.md" AGENTS.md
```

```
ls -la
```

> AGENTS.md 는 4장에서 채운 내용을 쓰는 것이 원칙입니다. 4장을 건너뛰었다면 templates\AGENTS_빈칸.md 를 먼저 채우고 여기에 넣습니다.

2단계. 저장소 만들고 훅 켜기 (10분)

[컨테이너]

```
git init
git config core.hooksPath .githooks
echo ".env" > .gitignore
```

이번 장에서는 **저장(커밋)을 마지막에 한 번만** 합니다. 지금 커밋하면 todo.py 가 없어서 검증이 실패하고 취소됩니다. 5장에서 본 장치라서 여기서는 하지 않습니다.

> 원본 테스트를 기준으로 남겨 둡니다. 나중에 테스트가 바뀌었는지 비교하려고 합니다.

3단계. 테스트 기준 읽기 (20분)

tests/test_todo.py 를 엽니다. 이 파일이 할 일 목록의 **약속**입니다.

[Windows]

```
# (Windows) tests\test_todo.py 을(를) VS Code 로 엽니다
```

아래 세 가지를 찾아 적습니다.

1. add_task 가 돌려주는 값은 무엇인가?
2. 제목이 공백뿐이면 무엇이 일어나야 하나?
3. 파일이 없을 때 load_tasks 는 무엇을 돌려주나?

이 세 답이 곧 AI 가 지켜야 할 기준입니다.

4단계. 지시서와 계획 확인하기 (15분)

TASK_todo.md 와 REQUIREMENTS.md 를 읽습니다. 지시서의 "고쳐도 되는 파일"은 무엇입니까? "고치면 안 되는 파일"은 무엇입니까?

OpenCode 를 엽니다.

[컨테이너]

```
opencode
```

■ [AI] OpenCode 입력 칸

```
TASK_todo.md 를 읽고 그 지시대로 작업해 줘. AGENTS.md 의 작업 순서를 따라, 먼저 계획만 말하고 내 승인을 기다려.
```

AI 의 계획을 아래 체크리스트로 검토합니다.

- ■ 고칠 파일이 todo.py 하나뿐인가?
- ■ tests/, verify.py, AGENTS.md 를 고치겠다는 말이 없는가?
- ■ 파일 저장 기능(R4)을 계획에 넣었는가?
- ■ 외부 패키지를 설치하겠다는 말이 없는가?

맞으면 진행 이라고 답합니다. 맞지 않으면 다음처럼 고치게 합니다.

■ [AI] OpenCode 입력 칸

```
계획 3번에 웹 화면이 들어 있어. TASK_todo.md 의 "하지 말 것"에 있어서 빼고 다시 말해 줘.
```

5단계. 구현 지켜보기와 검증 (25분)

AI 가 `todo.py` 를 만드는 동안 지켜봅니다. 명령을 실행하기 전에 물어보면, 무엇을 하려는지 읽고 허락합니다. 허락은 `once` 만 씁니다.

끝나면 `/exit` 로 나오고, 검증을 직접 실행합니다.

[컨테이너]

```
python verify.py
```

■ "검증 통과"가 나온다.

■ 테스트 개수가 줄지 않았다.

검증이 실패하면, 실패 출력을 AI 에게 붙여 주고 **원인부터 설명하게** 합니다. 테스트를 고쳐서 통과시키라고 하지 않습니다.

■ [AI] OpenCode 입력 칸

```
python verify.py 가 실패했어. 아래가 출력이야. 원인을 먼저 설명하고, tests/ 는 건드리지 말고 todo.py 만 고쳐 줘.
(여기에 실패 출력을 붙여 넣는다)
```

> 세 번 고쳐도 통과하지 못하면 멈추고, `git restore todo.py` 로 되돌린 뒤 지시서를 더 작게 나눠서 다시 시킵니다. (새 파일은 `git status` 로 확인해 직접 지웁니다.)

6단계. 기준이 바뀌지 않았는지 확인하기 (10분)

[컨테이너]

```
git status
```

■ 아래 비교 명령에 아무것도 나오지 않는다. (테스트 파일이 템플릿과 같다)

■ `git status` 에 `todo.py` 가 새 파일로 나온다. 다른 파일은 새로 생기거나 바뀌지 않았다.

[컨테이너]

```
diff tests/test_todo.py /tutorial/templates/tests/test_todo.py
```

비교 결과가 나오면 AI 가 기준을 고친 것입니다. 저장하지 않습니다. AI 에게 되돌리게 합니다. (`git status` 에서 `tests/` 가 "새 파일"로 보이는 것은 정상입니다. 아직 한 번도 저장하지 않았기 때문입니다.)

7단계. 직접 실행해 보기 (10분)

테스트가 통과해도, 사람이 직접 써 봐야 합니다. 파이썬 대화 모드로 함수를 불러 봅니다.

[컨테이너]

```
python
```

```
>>> from todo import add_task, complete_task, load_tasks, save_tasks
>>> tasks = []
>>> add_task(tasks, "우유 사기")
>>> complete_task(tasks, 1)
>>> tasks
>>> exit()
```

`tasks` 에 `done` 이 `True` 로 바뀐 것이 보이면 확인이 된 것입니다.

8단계. 설명할 수 있는지 확인하고 저장하기 (15분)

저장하기 전에, `todo.py` 를 위에서 아래로 읽으며 **세 문장으로 설명**해 봅니다.

1. 할 일 한 건은 어떤 모양의 자료인가?
2. 번호는 어떻게 정하는가?
3. 파일 저장과 읽기는 어떻게 하는가?

설명하지 못하는 부분이 있으면 AI 에게 묻습니다.

■ [AI] OpenCode 입력 칸

```
파일은 고치지 말고 설명만 해 줘. todo.py 의 add_task 에서 번호를 정하는 줄이 무슨 뜻이야? 초보자에게 두 문장으로 알려 줘.
```

설명을 이해했으면 저장합니다.

[컨테이너]

```
git add -A
git status
git commit -m "6장: 할 일 목록 완성 (R1~R4, 검증 통과)"
git log --oneline
```

`[pre-commit]` 이 통과하고 커밋이 한 줄 생기면 완료입니다.

9단계. 과정 기록 쓰기 (10분)

`my-todo` 폴더에 `기록.md` 를 만들고, 아래 틀을 채웁니다.

```
# 6장 기록

## 1. AI 에게 시킨 일 (한 문장)

## 2. AI 가 세운 계획 중 고치게 한 것 (없으면 "없음")

## 3. 막힌 것이나 물어본 것 (권한 프롬프트, 검증 실패 횟수)

## 4. 내 말로 설명한 코드 세 문장
1.
2.
3.

## 5. 새로 알게 된 것 한 가지

## 6. 아직 모르는 것 한 가지
```

[Windows]

```
# (Windows) 기록.md 을(를) VS Code 로 열어 고칩니다
```

3. 확인하기

■ `python verify.py` 가 "검증 통과"로 끝났다.
■ 테스트 파일을 템플릿과 비교했을 때 달라진 곳이 없다.
■ 계획 단계에서 범위 밖 기능을 하나 이상 빼게 했다.
■ 저장 커밋이 한 줄 생겼다.
■ `기록.md` 에 설명 세 문장을 썼다.

4. 막혔을 때

증상	해결
테스트가 계속 실패한다	실패한 테스트 이름과 `assert` 줄을 AI 에게 붙여 주고 원인부터 설명하게 한다.
AI 가 테스트를 고치려 한다	권한 설정(`opencode.jsonc`)이 막는다. 막히지 않으면 `tests/` 를 되돌린다(`git restore tests/`).
계획이 자꾸 범위를 넘는다	`TASK_todo.md` 의 "하지 말 것"을 가리키며 빼게 한다.
어디서부터 손대야 할지 모르겠다	`tests/test_todo.py` 의 테스트를 하나씩 통과시킨다. 한 번에 다 하지 않는다.

> 이 장에서는 **정답 코드를 제공하지 않습니다.** 답을 보기 전에 테스트와 지시서를 읽고, AI 와 대화하며 스스로 맞춰 가는 것이 목표입니다. 막히면 강사에게 물어보세요.

5. 확인 문제

1. `TASK_todo.md` 에서 "고치면 안 되는 파일"을 적은 이유는 무엇인가?
2. 이번 장에서 저장을 마지막에 한 번만 한 이유를 설명하시오.
3. 이번 장에서 **사람이 한 판단**을 세 가지 적으시오.
4. 테스트가 통과했는데도 직접 실행해 본 이유는 무엇인가?
5. 1장의 실험 A 와 이번 장을 비교했을 때, 달라진 점을 두 가지 쓰시오.

다음 단계

이것으로 1단계가 끝납니다. 2단계는 같은 흐름을 **FastAPI 웹 API**와 **파일 기반 저장**으로 넓혀 갑니다. 2단계 교안은 이 저장소의 `docs/stage2/` 에 추가될 예정입니다.

부록. 템플릿과 예제

수업에서 복사해 쓰는 파일을 그대로 실었다. 주석을 따라 읽는다.

A-1. 요구사항 빈칸 틀 (2장)

```
# 요구사항 (빈칸을 채우는 틀)

> 2교시에 쓰는 틀이다. 무엇을 만들지 "이렇게 부르면 → 이런 결과" 형식으로 적는다.
> "어떻게 만들지"는 적지 않는다. 그것은 AI 가 정한다.

## 한 문장 요약
(무엇을 하는 프로그램인지 한 문장으로)

## 기능 요구사항

### R1. (기능 이름)
- 이렇게 하면: (입력 또는 사용 방법)
- 이런 결과가 나온다: (결과)
- 잘못 쓰면: (잘못된 입력일 때 무엇이 나오는가)

### R2. (기능 이름)
- 이렇게 하면:
- 이런 결과가 나온다:
- 잘못 쓰면:

## 만들지 않는 것
- (이번에 만들지 않을 기능을 두세 개)

---

## 체크리스트 (다 쓴 뒤 스스로 확인)
- [ ] 두 사람이 읽으면 같은 것을 만들 수 있는가?
- [ ] 모든 기능에 "잘못 쓰면" 항목이 있는가?
- [ ] 결과가 "잘 된다"처럼 흐릿하지 않고, 눈으로 확인할 수 있는가?
- [ ] "만들지 않는 것"이 적혀 있는가?
```

A-2. 작업 지시서 빈칸 틀 (3장)

```
# 작업 지시서 (빈칸을 채우는 틀)

> 3교시에 쓰는 틀이다. AI 에게 일을 시키기 전에, 아래 여섯 칸을 채워서 건넨다.
> 한 줄로 "프로그램 만들어 줘"라고 시키지 않는다.

## 1. 목표
(이 작업이 끝나면 무엇이 되어 있어야 하는가. 한두 문장)

## 2. 요구사항
(요구사항 문서의 번호. 예: R1, R2)

## 3. 범위
- 고쳐도 되는 파일:
- 고치면 안 되는 파일:

## 4. 하지 말 것
- (AI 가 덧붙이기 쉬운 것을 적는다. 예: 파일 저장 기능은 만들지 않는다)
- (새로 설치할 패키지가 있다면 적는다. 없으면 "새 패키지를 설치하지 않는다")

## 5. 완료 조건
- [ ] `python verify.py` 가 "검증 통과"로 끝난다.
- [ ] (눈으로 확인할 수 있는 조건을 한 줄씩)

## 6. 진행 방법
먼저 계획만 말한다. 내가 "진행"이라고 하기 전에는 파일을 고치지 않는다.
끝나면 무엇을 바꿨는지, 어떤 명령으로 확인했는지 보고한다.

---

## 좋은 지시와 나쁜 지시 비교
| 나쁜 지시 | 좋은 지시 |
|---|---|
| "할 일 앱 만들어 줘" | "R1~R3 을 구현한다. 파일 저장은 하지 않는다(메모리에만)." |
| "에러 고쳐 줘" | "`python verify.py` 결과를 붙인다. 원인을 먼저 설명하고 고쳐 줘." |
| "알아서 잘 해 줘" | "두 방법이 있으면 장단점을 말하고 내 선택을 기다려 줘." |
| "테스트 통과시켜 줘" | "todo.py 를 고쳐서 통과시켜 줘. tests/ 는 고치지 않는다." |
```

A-3. 규칙 문서 빈칸 틀 (4장)

```
# AGENTS.md — 작업 규칙 (빈칸을 채우는 틀)

> 4교시에 채운다. AI 에이전트(OpenCode)는 이 프로젝트에서 일을 시작할 때 이 파일을 먼저 읽는다.
> 규칙은 "부탁"이다. 잠금장치(opencode.jsonc)와 함께 있어야 힘이 생긴다.
> 빈칸 `[작성]` 을 채우고, 아래 "꼭 들어가야 할 것" 목록을 모두 체크한다.

## 1. 이 프로젝트는 무엇인가
- 목표: [작성]
- 사용자: [작성] (이 코드를 읽을 사람. 예: 프로그래밍을 처음 배우는 사람)

## 2. 파일 나누기
| 파일 | 고쳐도 되는가 |
|---|---|
| todo.py | [작성] |
| tests/ | [작성] |
| AGENTS.md, opencode.jsonc, verify.py | [작성] |

## 3. 일하는 순서
1. [작성] (먼저 무엇을 하는가)
2. [작성] (계획을 말하고 무엇을 기다리는가)
3. [작성] (구현)
4. [작성] (검증 방법)
5. [작성] (보고 방법)

## 4. 하지 말 것
- [작성] (커밋과 푸시는 누가 하는가)
- [작성] (테스트에 대해 무엇을 금지하는가)
- [작성] (새 패키지나 인터넷 사용에 대해)

## 5. 코드 규칙
- [작성] (주석은 어떤 언어로, 어디에 달 것인가)
- [작성] (이름 짓기, 한 줄에 한 가지 일 등)

## 6. 검증
- 완료라고 말하기 전에 [작성] 명령을 실행한다.
- 검증이 실패하면 [작성] (몇 번까지 고쳐 보고, 그다음에는 무엇을 하는가)

## 7. 보고 형식
```

[한 일] [바뀐 파일] [검증 결과] (python verify.py 의 마지막 줄) [확인 방법] [남은 일]

```
---

## 꼭 들어가야 할 것 (체크리스트)
- [ ] 계획을 먼저 말하고, 사람의 "진행"을 기다린다는 문장
- [ ] 테스트 파일을 고치지 않는다는 문장
- [ ] 커밋과 푸시는 하지 않는다는 문장
- [ ] `python verify.py` 로 검증한다는 문장
- [ ] 바뀐 파일과 확인 방법을 보고한다는 문장
```

A-4. 권한 설정 완성본 (4장)

```
// ============================================================
// opencode.jsonc - AI 에이전트(OpenCode) 권한 설정  [완성본]
//
// 4교시에 이 파일을 프로젝트 맨 위 폴더로 복사한다.
// 규칙(AGENTS.md)은 "부탁"이고, 이 파일은 "잠금장치"다.
//
// 읽는 법
//   action   : 무엇을 하려는가  (shell = 명령 실행, edit = 파일 고치기, read = 파일 읽기)
//   resource : 무엇에 대해서인가 (* 는 "아무 글자나")
//   effect   : allow(그냥 허용) / ask(나에게 물어봄) / deny(금지)
//
// 위에서 아래로 읽으며, 여러 규칙에 걸리면 "가장 아래 규칙"이 이긴다.
// 그래서 넓은 규칙을 위에, 예외를 아래에 쓴다.
// ============================================================
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/anthropic/claude-haiku-5.5",
  "share": "disabled",

  "permissions": [
    // ---------- 1. 명령 실행 ----------
    // 기본: 모든 명령은 실행 전에 나에게 묻는다.
    { "action": "shell", "resource": "*", "effect": "ask" },
    // 검증과 읽기 명령은 묻지 않는다.
    { "action": "shell", "resource": "python verify.py", "effect": "allow" },
    { "action": "shell", "resource": "python -m pytest *", "effect": "allow" },
    { "action": "shell", "resource": "git status *", "effect": "allow" },
    { "action": "shell", "resource": "git diff *", "effect": "allow" },
    { "action": "shell", "resource": "git log *", "effect": "allow" },
    // 저장과 올리기는 사람만 한다. 에이전트는 금지.
    { "action": "shell", "resource": "git commit *", "effect": "deny" },
    { "action": "shell", "resource": "git push *", "effect": "deny" },
    { "action": "shell", "resource": "git reset *", "effect": "deny" },

    // ---------- 2. 파일 고치기 ----------
    // 규칙, 권한, 검증 스크립트, 테스트 기준은 에이전트가 고치지 못한다.
    { "action": "edit", "resource": "*AGENTS.md", "effect": "deny" },
    { "action": "edit", "resource": "*opencode.jsonc", "effect": "deny" },
    { "action": "edit", "resource": "*verify.py", "effect": "deny" },
    { "action": "edit", "resource": "*/tests/*", "effect": "deny" },
    { "action": "edit", "resource": "*pre-commit", "effect": "deny" },

    // ---------- 3. 비밀 값 ----------
    { "action": "read", "resource": "*.env", "effect": "deny" },

    // ---------- 4. 인터넷 ----------
    { "action": "webfetch", "resource": "*", "effect": "ask" },
    { "action": "websearch", "resource": "*", "effect": "ask" }
  ]
}
```

A-5. 검증 스크립트 (5장)

```
"""
verify.py - 검증 스크립트 (하네스의 "검증" 담당)

사용법 (프로젝트 폴더에서):  python verify.py
  - 테스트가 모두 통과하면 "검증 통과"를 출력하고 끝난다.
  - 하나라도 실패하면 "검증 실패"를 출력하고, 종료 코드 1 로 끝난다.

커밋 직전에 이 파일이 자동으로 실행된다(.githooks/pre-commit).
실패하면 커밋이 취소된다. 그래서 "테스트를 통과한 것만 저장된다"는 규칙이 사람의 기억이 아니라 도구로 지켜진다.
"""

import subprocess
import sys

# 파이썬으로 pytest 를 실행한다. 결과(통과/실패)는 pytest 가 정한다.
result = subprocess.run([sys.executable, "-m", "pytest", "-q"])

if result.returncode != 0:
    print("")
    print("검증 실패: 위의 오류를 고친 뒤 다시 실행하세요.")
    sys.exit(1)  # 0 이 아닌 값으로 끝나야 커밋이 취소된다

print("")
print("검증 통과: 이제 저장(commit)해도 됩니다.")
```

A-6. 커밋 전 검증 훅 (5장)

```
#!/bin/sh
# ============================================================
# pre-commit - 커밋 직전에 git 이 자동으로 실행하는 스크립트
#
# verify.py 가 실패하면(종료 코드 1) 이 스크립트도 1 로 끝나고,
# git 은 커밋을 만들지 않는다.
#
# 켜는 법 (프로젝트 폴더에서 한 번만):
#   git config core.hooksPath .githooks
# ============================================================

echo "[pre-commit] 저장하기 전에 검증을 실행합니다..."

python verify.py
result=$?

if [ $result -ne 0 ]; then
    echo ""
    echo "[pre-commit] 검증 실패: 커밋을 취소했습니다."
    exit 1
fi
```

A-7. 할 일 목록 요구사항 (6장)

```
# 요구사항: 할 일 목록 (6교시 미니 프로젝트)

## 한 문장 요약
할 일을 적고, 목록으로 보고, 끝낸 일을 표시하고, 지울 수 있는 명령줄 프로그램. 목록은 파일에 저장되어 프로그램을 껐다 켜도 남는다.

## 할 일 한 건의 모양
| 항목 | 설명 |
|---|---|
| `id` | 1, 2, 3 … 순서대로 붙는 번호. 지워도 다시 쓰지 않는다. |
| `title` | 할 일의 내용. 공백만 있으면 안 된다. |
| `done` | 끝났으면 `True`, 아직이면 `False`. 처음에는 `False`. |

## 기능 요구사항

### R1. 할 일 추가
- `add_task(tasks, title)` 을 부르면 새 할 일을 목록 끝에 넣고, 그 할 일을 돌려준다.
- 제목이 공백뿐이면 `ValueError` 를 일으킨다.

### R2. 완료 표시
- `complete_task(tasks, task_id)` 을 부르면 그 번호의 할 일을 `done = True` 로 바꾼다.
- 그 번호가 없으면 `ValueError` 를 일으킨다.

### R3. 삭제
- `delete_task(tasks, task_id)` 을 부르면 그 번호의 할 일을 목록에서 뺀다.
- 그 번호가 없으면 `ValueError` 를 일으킨다.

### R4. 파일 저장과 읽기
- `save_tasks(tasks, path)` 은 목록을 JSON 파일로 저장한다.
- `load_tasks(path)` 은 JSON 파일을 읽어 목록을 돌려준다.
- 파일이 아직 없으면 빈 목록 `[]` 을 돌려준다. 오류를 내지 않는다.

### R5. 명령줄에서 쓰기 (선택 사항)
- `python todo.py` 뒤에 `add`, `list`, `done`, `delete` 를 붙여 쓸 수 있다.
- 목록 보기는 번호, 완료 여부(`[x]` 또는 `[ ]`), 제목을 한 줄씩 보여 준다.

## 만들지 않는 것
- 마감일, 우선순위, 검색, 정렬
- 웹 화면, 데이터베이스, 여러 사람이 함께 쓰는 기능
```

A-8. 할 일 목록 작업 지시서 (6장)

```
# 작업 지시서: 할 일 목록 만들기 (6교시)

## 1. 목표
할 일을 추가하고, 완료 표시하고, 지우고, 파일에 저장할 수 있는 `todo.py` 를 만든다.

## 2. 요구사항
`REQUIREMENTS.md` 의 R1~R5 를 구현한다.

## 3. 범위
- 새로 만드는 파일: `todo.py`
- 고쳐도 되는 파일: `todo.py` 하나뿐이다.
- 고치면 안 되는 파일: `tests/` 폴더 전체, `verify.py`, `AGENTS.md`, `opencode.jsonc`

## 4. 하지 말 것
- 웹 화면, 데이터베이스, 외부 패키지(설치가 필요한 것)를 쓰지 않는다. 파이썬 기본 기능만 쓴다.
- 요구사항에 없는 기능(검색, 정렬, 마감일)을 덧붙이지 않는다.
- 테스트 파일의 기대값을 바꾸거나 테스트를 건너뛰게 만들지 않는다.

## 5. 완료 조건
- [ ] `python verify.py` 가 "검증 통과"로 끝난다.
- [ ] `tests/test_todo.py` 의 테스트가 모두 통과한다. (테스트 개수는 줄지 않았다)
- [ ] `todo.py` 의 모든 함수에 한국어 주석이 있다.
- [ ] `git diff tests/` 에 아무 변화가 없다.

## 6. 진행 방법
`AGENTS.md` 의 작업 절차를 따른다. 먼저 계획을 말하고, 내가 "진행"이라고 하면 구현한다.
끝나면 `AGENTS.md` 의 보고 형식으로 보고한다.
```

A-9. 할 일 목록 테스트 (6장)

```
"""
test_todo.py - 할 일 목록 프로그램(todo.py)이 지켜야 할 약속

6교시에 AI 에이전트는 이 테스트를 통과하도록 todo.py 를 만든다.
이 파일은 사람이 쓴 "기준"이다. AI 는 이 파일을 고칠 수 없다(opencode.jsonc 에서 막는다).

이 파일이 보여 주는 개념
  1) 기준(테스트)은 사람이 정하고, 구현(todo.py)은 AI 가 맡는다.
  2) 정상 경우와 잘못된 경우를 모두 약속한다.
  3) 파일 저장은 임시 폴더(tmp_path)에서만 한다. 내 진짜 파일을 건드리지 않는다.
"""

import pytest

from todo import add_task, complete_task, delete_task, load_tasks, save_tasks


def test_add_task_creates_new_task():
    tasks = []
    task = add_task(tasks, "우유 사기")
    assert task == {"id": 1, "title": "우유 사기", "done": False}
    assert tasks == [task]  # 목록에도 들어가야 한다


def test_add_task_numbers_go_up():
    tasks = []
    add_task(tasks, "첫째")
    second = add_task(tasks, "둘째")
    assert second["id"] == 2


def test_add_task_rejects_blank_title():
    # 공백만 있는 제목은 받지 않는다.
    with pytest.raises(ValueError):
        add_task([], "   ")


def test_complete_task_marks_done():
    tasks = []
    add_task(tasks, "운동하기")
    complete_task(tasks, 1)
    assert tasks[0]["done"] is True


def test_complete_task_not_found():
    with pytest.raises(ValueError):
        complete_task([], 99)


def test_delete_task_removes_it():
    tasks = []
    add_task(tasks, "지울 일")
    delete_task(tasks, 1)
    assert tasks == []


def test_delete_task_not_found():
    with pytest.raises(ValueError):
        delete_task([], 99)


def test_save_and_load_roundtrip(tmp_path):
```

```
    path = tmp_path / "tasks.json"  # 임시 폴더 안의 파일만 쓴다
    tasks = []
    add_task(tasks, "저장할 일")
    save_tasks(tasks, path)
    assert load_tasks(path) == tasks


def test_load_missing_file_returns_empty_list(tmp_path):
    # 파일이 아직 없으면 빈 목록을 돌려준다(오류가 나지 않는다).
    assert load_tasks(tmp_path / "없는파일.json") == []
```

A-10. 계산기 예제 (4·5장)

```
"""
calc.py - 아주 작은 계산 함수 모음 (1단계 연습용)

이 파일이 보여 주는 개념
  1) 함수는 "입력 → 계산 → 결과"에 이름을 붙인 것이다.
  2) 잘못된 입력(0으로 나누기)은 조용히 틀린 답을 내지 않고 오류로 알린다.

5교시 연습: multiply(곱하기) 함수가 아직 없다. 테스트를 먼저 실행하면 실패한다(빨강).
          AI 에게 multiply 를 추가하게 해서 테스트를 통과시킨다(초록).
"""


def add(a: float, b: float) -> float:
    """두 수를 더한다."""
    return a + b


def subtract(a: float, b: float) -> float:
    """a 에서 b 를 뺀다."""
    return a - b


def divide(a: float, b: float) -> float:
    """a 를 b 로 나눈다. b 가 0 이면 ValueError 오류를 일으킨다."""
    # 0 으로 나누면 계산할 수 없으므로 미리 막는다. 이 검사가 없으면 프로그램이 멈춘다.
    if b == 0:
        raise ValueError("0으로 나눌 수 없습니다.")
    return a / b
```

A-11. 계산기 테스트 (5장)

```
"""
test_calc.py - calc.py 가 약속대로 동작하는지 확인하는 테스트

이 파일이 보여 주는 개념
  1) 테스트는 "이 값을 넣으면 이 값이 나와야 한다"를 코드로 적은 약속이다.
  2) assert 뒤의 식이 참이면 통과, 거짓이면 실패다.
  3) 정상 경우뿐 아니라 잘못된 경우(0으로 나누기)도 약속에 넣는다.
"""

import pytest

from calc import add, divide, multiply, subtract


def test_add():
    assert add(2, 3) == 5


def test_subtract():
    assert subtract(10, 4) == 6


def test_divide():
    assert divide(9, 3) == 3


def test_divide_by_zero_raises_error():
    # 0 으로 나누면 ValueError 가 나와야 한다.
    with pytest.raises(ValueError):
        divide(1, 0)


def test_multiply():
    # 5교시에 AI 가 multiply 를 추가하면 이 테스트가 통과한다.
    assert multiply(4, 5) == 20
```